Driver Pulse

Real-Time Safety & Earnings Intelligence for Uber Drivers

Team: Hamoya Buthiriyo · Madhur · Shobhada Gupta
Date: March 10, 2026
Stack: Flutter · Firebase · Pure Python (Heuristics - ML)
Philosophy: Edge first. No extra hardware. Just the phone.

Contents

1. The Problem Uber Hasn't Solved Yet
2. Our Solution in 30 Seconds
3. System Architecture
4. Motion Classifier — Math & Thresholds
5. Audio Classifier — Why 48 Kilos Fails
6. Conflict Detection — Dual-Signal Amplification
7. Earnings Intelligence Engine
8. Edge Cases We Caught
9. Scalability — From 1 Driver to 5 Million
10. Why This Matters to Uber (Business Impact)

1. The Problem Uber Hasn't Solved Yet

Uber has **5 million drivers** globally. But the platform has no real-time safety feedback loop for drivers during a trip. Everything is post-hoc: ratings come after the ride, safety reports come days later.

PROBLEM	CURRENT STATE	DRIVER PAIN POINT
Harsh braking	Not detected. No feedback.	Classified per-second Flagged live.
In-cabin conflict / road rage	Only reported via rider complaint	Detected via sustained audio duration
"Am I on pace to hit my earnings goal?"	No answer until shift ends	Live velocity tracking vs adaptive threshold
False positives (potholes = "harsh braking")	Existing telematics can't distinguish	Gyro spike plus Mahal 95% of false positives
Bandwidth at scale	Raw sensor upload = 2.5 TB/hr globally	Edge classification = **20 GB/hr** (99% reduction)

> **WHY THIS MATTERS TO UBER**
> Uber's largest operational cost after driver payouts is **support & safety incidents**. A system that catches conflicts and risky driving in real-time — before a rider rating or an accident report — directly reduces support tickets, insurance claims, and churn.

2. Our Solution in 30 Seconds

3	O(1)	<500B
PHONE SENSORS USED	PER-SAMPLE COMPUTE COMPLEXITY	TOTAL COMPANION STATE
99%	**4**	**0**
BANDWIDTH SAVINGS	SEVERITY LEVELS	EXTRA HARDWARE NEEDED

Driver Pulse runs **entirely on the driver's existing phone**. It reads the accelerometer, microphone, and GPS at 1Hz, classifies each sample in O(1) time, and only sends flagged events to the server. The driver sees a live dashboard with safety flags and earnings coaching.

3. System Architecture

> **KEY INSIGHT**
> Every classifier is a **stateless pure function** — no rolling windows, no ML inference, no matrix operations. This means it can run on a $50 Android phone without draining the battery, and it's trivially portable to iOS, Kotlin, or even embedded C.

4. Motion Classifier — Math & Thresholds

4.1 Gravity Compensation

A phone mounted on a dashboard is tilted. Raw Z-axis reads ~9.7kg, not 9.81g. We subtract this offset so only real motion registers.

```
lowpass(raw_accel, raw_accel) = beta*raw
lowpass(raw_accel) + (1-beta)*lowpass(raw_accel)
gravity_estimate = lowpass(raw_accel)
linear_accel = raw_accel - gravity_estimate
```

> **JUDGE CODE HANDLE**
> GPS mount repeatedly **2.5 hours**. We use speed > 2.5 m/s as the "stationary" threshold. Every trip also starts with 2 warmup samples at speed 0 (simulating passenger pickup), guaranteeing calibration always succeeds.

4.2 Braking & Cornering Thresholds

Derived from dataset percentile analysis (not guessed).

EVENT	AXIS	X-FORCE RATE	SPEED-DELTA RATE	SCORE
Emergency stop	Y (braking)	< -6.5g	< -15 km/h	**1.00**
Harsh brake	Y	< -4.5g	< -10 km/h	**0.82**
Moderate brake	Y	< -3.5g	< -5 km/h	**0.48**
Soft brake	Y	< -2.5g	none	0.20
Harsh corner	X (lateral)	±3.5g	—	**0.85**
Moderate corner	X	±2.5g	—	**0.50**

4.3 Speed-Delta Validation — The Key Innovation

```
speed_delta = current_speed - previous_speed

IF (accel_y) > threshold AND speed_delta < required_drop:
    "I feel a jolt AND my car slowed down" (score 0.85)
    → NOT a braking event!

This single check eliminates ~95% of false positives from potholes & road bumps.
```

> **WHY THIS MATTERS TO UBER**
> Rider trusts now an average of **20 platform per km** in cities like Mumbai. Without speed-delta validation, every pothole looks like a hard brake to the accelerometer. This gate is what makes the system usable in the real world, not just in a lab.

> **JUDGE CODE HANDLE**
> **Soft brake (-2.5g) intentionally stays the speed-delta gate.** At this low G-force, the deceleration is gentle enough to occur without a dramatic speed drop (e.g., rolling to a stop at a signal). Gating it would produce false negatives.

5. Audio Classifier — Why dB Alone Fails

5.1 The Data Told Us Something Surprising

We expected louder = more dangerous. The data disagreed.

AUDIO CLASS	DB RANGE IN DATASET	SUSTAINED DURATION (MEAN)
Quiet	50 – 60 dB	0 sec
Normal	52 – 65 dB	0 sec
Conversation	55 – 67 dB	0 sec
Loud	55 – 88 dB	0 sec
Very Loud	55 – 98 dB	79 sec
Argument	58 – 98 dB	**163 sec**

> **KEY INSIGHT**
> **dB ranges overlap completely.** A "quiet" cabin can reach 92 dB, an "argument" can reach 58 dB. The only reliable discriminator is sustained_duration. Loud music blasts for a few seconds then stops. This is the key insight that separates our classifier from naive dB-threshold approaches.

5.2 Classification Rules

```
IF sustained_sec > 90 AND db_baseline > 10 → "argument" (score 0.92)
IF sustained_sec > 30 → "very_loud" (score 0.65)
IF sustained_sec > 0 → "elevated" (score 0.35)
ELSE → "normal" (score 0.05)
```

> **JUDGE CODE HANDLE**
> **Pure Python heuristics.** The classifier averages 40 readings from the first 60 seconds to set a per-trip baseline. This means a rickshaw driver with a naturally noisy cabin doesn't get false "argument" classifications — the argument gate requires dB to be 10+ above that specific car's baseline.

6. Conflict Fusion — Dual-Signal Amplification

6.1 Fusion Formula

```
conflict_score = (motion_score * 0.55) + (audio_score * 0.45)

Dual-signal amplifier:
IF motion_score > 0.3 AND audio_score > 0.4:
    conflict_score = min(conflict_score * 1.3, 1.0)
```

Why 55/45? Motion is more mechanically dependable and more reliably measurable than audio. But audio captures interpersonal conflict (road rage, argumenta) that motion can't see. The weights reflect this asymmetry.

6.2 Severity Classification

SEVERITY	SCORE RANGE	REAL-WORLD EXAMPLE	ACTION
HIGH	≥ 0.70	Hard brake during verbal argument	Immediate alert
MEDIUM	≥ 0.40	Emergency stop at intersection	Flagged for review
LOW	≥ 0.25	Passenger on loud phone call	Logged for patterns
SAFE	< 0.25	Normal driving	Discarded (95% of samples)

6.3 Why the Amplifier Matters

SCENARIO	MOTION	AUDIO	WITHOUT AMPLIFIER	WITH AMPLIFIER
Hard brake only	0.82	0.00	0.451 → MEDIUM	0.451 → MEDIUM
Argument only	0.00	0.92	0.414 → LOW	0.414 → LOW
Brake + argument	0.82	0.92	0.865 → HIGH	**1.00 → HIGH** ↑

> **WHY THIS MATTERS TO UBER**
> **Single-channel events are common and usually benign** (a hard brake for a stop dog, a loud radio). But when both motion AND audio fire simultaneously, something genuinely dangerous is happening. The amplifier ensures these dual-signal events are never under-classified.

7. Earnings Intelligence Engine

Drivers don't just want safety — they want to know: *"Am I going to hit my earnings goal today?"*

7.1 The Math

```
STEP  FORMULA                                          PURPOSE
1. Target velocity   v_target = target_earnings /      $/hour needed
                     target_hours
2. Expected h/r      expected = v_target / current_hourly   Where should I be?
3. Early-shift       scale = 0.8 + 0.2 * (hours / 2.5)      Don't panic the driver
   penalty           remaining = 1 - scale * (hours / 8)    at hour 1
4. Earnings-gap      gap = current_earnings - expected      * ahead or behind
5. Dynamic           adj_thresh = stress*(target - 1) *     Personalized boundary
   threshold         sensitivity
                     rel_thresh = baseline* (1.0 -          Stress lowers
                     threshold * max(adj, .1))              threshold
6. Status            gap > rthreshold → "ahead"             Clear, actionable labels
                     gap < -rthreshold AND hours > 1.0 →
                     "behind"
                     else → "on_track"
7. Projection        projected = current + remaining_hrs *  End-of-shift forecast
                     velocity
                     clamped to [0, target * 2.0]
```

> **JUDGE CODE HANDLE**
> **Early shift scheduling uses a continuous ramp, not a hard cutoff.** The old approach (multiply expected by 0.8 if hours < 1) created a discontinuity at exactly 1 hour — a driver at 0.99 hours got a different target than one at 1.01 hours suddenly "ok ok" with no change in actual earnings. The ramp fixes this.

7.2 Alert Tone Adapts to Severity

STATUS	% BEHIND	MESSAGE
Ahead	—	"Great pace! You're ahead of target."
On track	—	"You're on track. Keep going."
At risk (>10%)	< 10%	"You're slightly behind pace. A couple of good trips can catch you up."
At risk (>10%)	> 10%	"You're significantly behind pace. Consider moving to a high-demand area."

> **WHY THIS MATTERS TO UBER**
> This isn't just information — it's **behavioral nudging**. A driver who knows they're 15% behind at 2pm can move to a surge zone. Without this, they find out at 10pm that they missed their goal — too late to do anything.

8. Edge Cases We Caught

The difference between a hackathon demo and a real system is edge cases. Here's every one we found and fixed:

#	EDGE CASE	WHAT WOULD BREAK	FIX	SEVERITY
1	Trip ends at 8th tenth, next starts at 0	Speed delta = -60 → phantom emergency stop, or first samples of new trip	Reset: prev_speed = 0 at each trip boundary	**Critical**
2	Load trip ending — next trip starts carries over — false emergency stop	suddenly jolt, bad data	Reset suddenly_bad_sec = 0 on "emergency" classification	**Critical**
3	GPS repeats 2.5 hours — looks like driver parked	Gravity calibration requires speed > 2.5 m/s — never triggers (continuous distribution)	Changed to: speed > 0.5 m/s	**Critical**
4	Pothole at 40 km/h — looks like hard brake	Classified as emergency_stop, without speed-delta gate	Speed-delta validation: no speed drop → road_event	**High**
5	Driver at 0.99 hrs — 1.01 hrs	Hard cutoff: status flips from "behind" to "ok ok" with no warning or change	Smooth linear ramp (0.8 → 1.0 over first hour)	**High**
6	Very short trip (5 minutes) — 1 event — looks like 100% baseline → 78 dB	Anomaly injection at start — 1 event skewed array	Bounds guard: 20 event 1.5 × mid_baseline	**High**
7	Sensor jumps from "elevated" to "very_loud"	Discrete bucket jumps — false escalation	Per-trip baseline calibration from first 60 seconds	Medium
8	Target_hours = 0	Division by zero in velocity calculation	Guard: fallback to target_earnings / 8 + warning log	Medium
9	Lucky first trip — projected earnings = 10x target	Unrealistic projection displayed to driver	Capped at target * 2.0	Medium
10	Phone orientation changes mid-trip	Gravity baseline becomes wrong	Low-pass calibration from stationary samples; fallback to dataset mean	Low

9. Scalability — From 1 Driver to 5 Million

9.1 The Bandwidth Problem

APPROACH	DATA PER DRIVER/HR	5M DRIVERS/HR	ANNUAL COST (AWS S3)
Raw sensor stream	~500 KB	**2.5 TB**	~$575,000
Driver Pulse (Haggar only)	**~2-5 KB**	**20 GB**	**~$5,750**

That's a 99% reduction. The math is simple: 95%+ of driving samples are (all) and never leave the phone.

9.2 Why Edge First Works

PROPERTY	OUR CLASSIFIERS	TYPICAL ML APPROACH
Complexity per sample	O(1) — comparisons only	O(n²) — matrix multiply
Memory footprint	< 500 bytes	50-500 MB (model weights)
Battery impact	Negligible	10-20% drain/hour
Dependencies	Zero (pure arithmetic)	TensorflowPy/Torch runtime
Latency	< 1ms per sample	50-200ms per inference
Portability	Any language, any platform	Specific runtime required

9.3 Backend Scaling

TIER	DRIVERS	PLANNED EVENTS/SEC	BACKEND INSTANCES	INGESTION BANDWIDTH
City pilot (Mumbai)	10,000	~100	2-3	~10 KB/s
Country (India)	500,000	~5,000	15-20	~2.5 MB/s
Global	5,000,000	~200,000	100+	~25 MB/s

> **KEY INSIGHT**
> The backend is **stateless per request**. Each classifier event is self-contained — no session state, no rolling windows, no cross-driver dependencies. This means horizontal scaling is trivial: just add more Kubernetes pods behind a load balancer.

10. Why This Matters to Uber

10.1 Direct Business Impact

USER METRIC	HOW DRIVER PULSE HELPS	ESTIMATED IMPACT
Support tickets per 1000 rides	Real-time conflict detection catches issues before they become complaints	−15–20% tickets
Insurance claim rate	Drivers aware of harsh braking patterns change behavior over time	−10–15% claims
Driver retention	Earnings coaching reduces "I don't know if this is worth it" anxiety	+5–8% retention
Platform safety ratings	Proactive safety monitoring lost just reactive ratings	Brand trust ↑
Data infrastructure cost	Edge classification = 99% less data to ingest and store	~$500K/yr saved

10.2 What Makes This Different from Existing Solutions

FEATURE	UBER'S CURRENT SYSTEM	COMPANION TELEMATICS	DRIVER PULSE
Safety detection	Post-ride ratings	Post-trip reports	Per-second, live
Audio conflict detection	✗ None	✗ None	✓ Sustained duration based
Pothole filtering	✓ Yes (uses gyro + brake)	△ Partial (using accelerometer)	✓ Speed-delta gate
Earnings coaching	✗ None real-time	✗ Not their domain	✓ Dynamic thresholds
Extra hardware required	✗ Dedicated OBD dongle ($50+)	✗ Dedicated OBD dongle	✓ None (phone only)
Edge classification	✗ None upload	✗ None upload	✓ Phone-side, O(1)
Dual-signal fusion	✗ No	✗ No	✓ Motion + Audio

10.3 Deployment Path

PHASE	TIMELINE	SCOPE	VALIDATION
1. Pilot	Month 1-2	100 drivers in Mumbai	Compare flag accuracy vs manual review
2. City rollout	Month 3-4	10,000 drivers in Mumbai	Measure ticket reduction, driver NPS
3. Country	Month 5-8	All India drivers	Insurance claim rate, retention metrics
4. Global	Month 9–12	All markets	Per-market threshold calibration